Intro to AI for Medicine

Lec 4: Training

Last Time

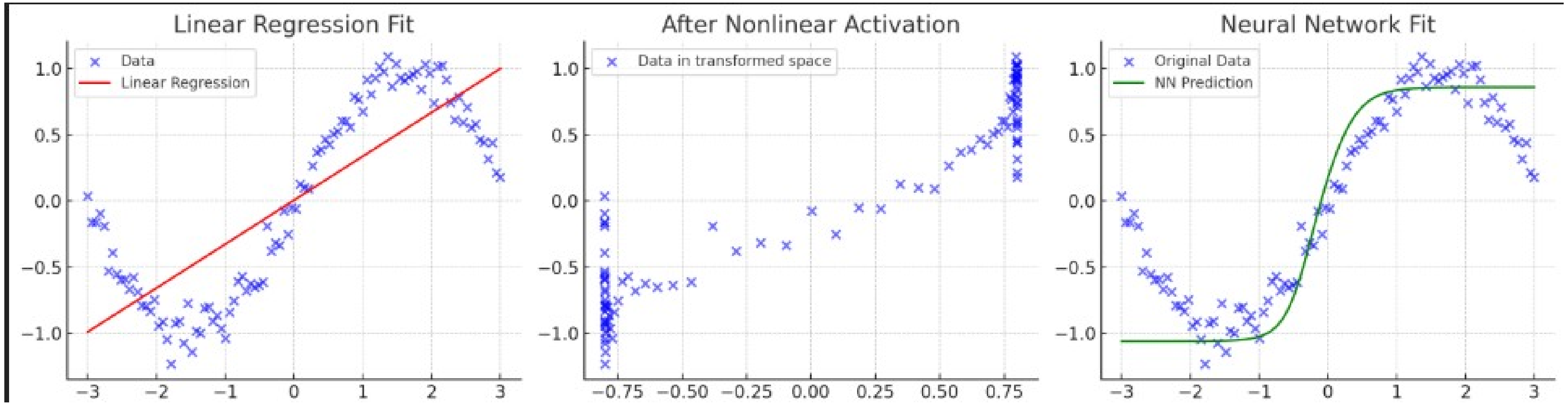
- Learned about loss functions, finishing neural net structure
- Defined regression, classification (including multiclass)
- Associated final activation with loss function and problem type
- Discussed some metrics for measuring how good our networks are

Agenda

- Overfitting v underfitting
- Internal/external validity
- Hyperparameters
- Revisit GD
- Ensembling
- Case study – afib ablations w/wo AI tool RCT

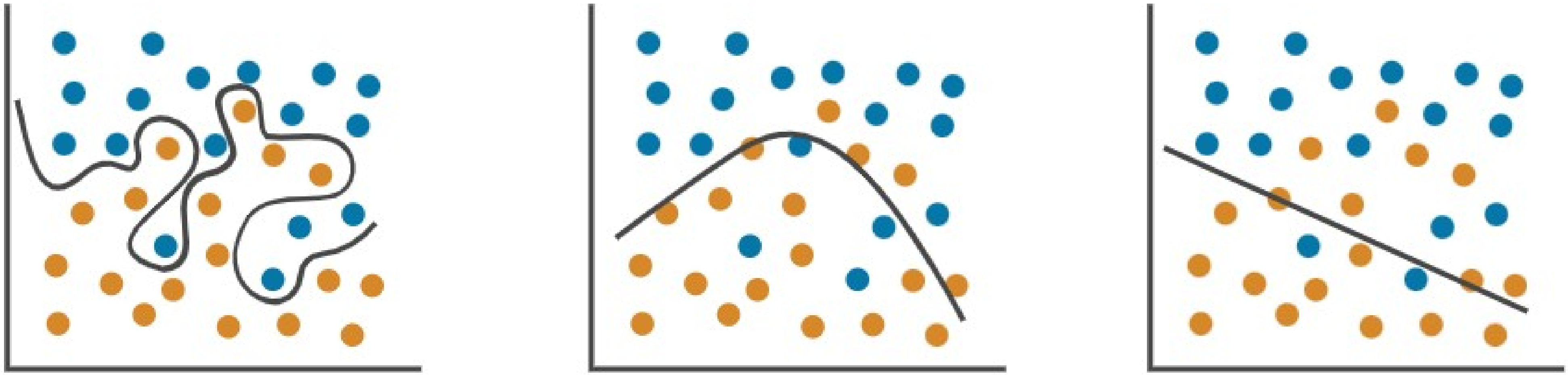
So Far…

- We've seen plots like below, and assumed a good fit
- How can we leverage metrics to tell us if we're really fitting the trend?
- How do we know if our model will work on other populations?

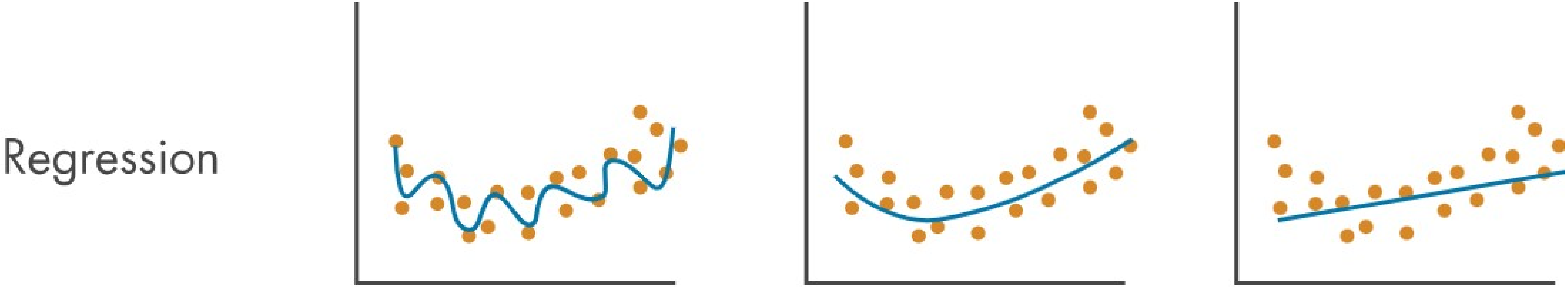


Fitting Data

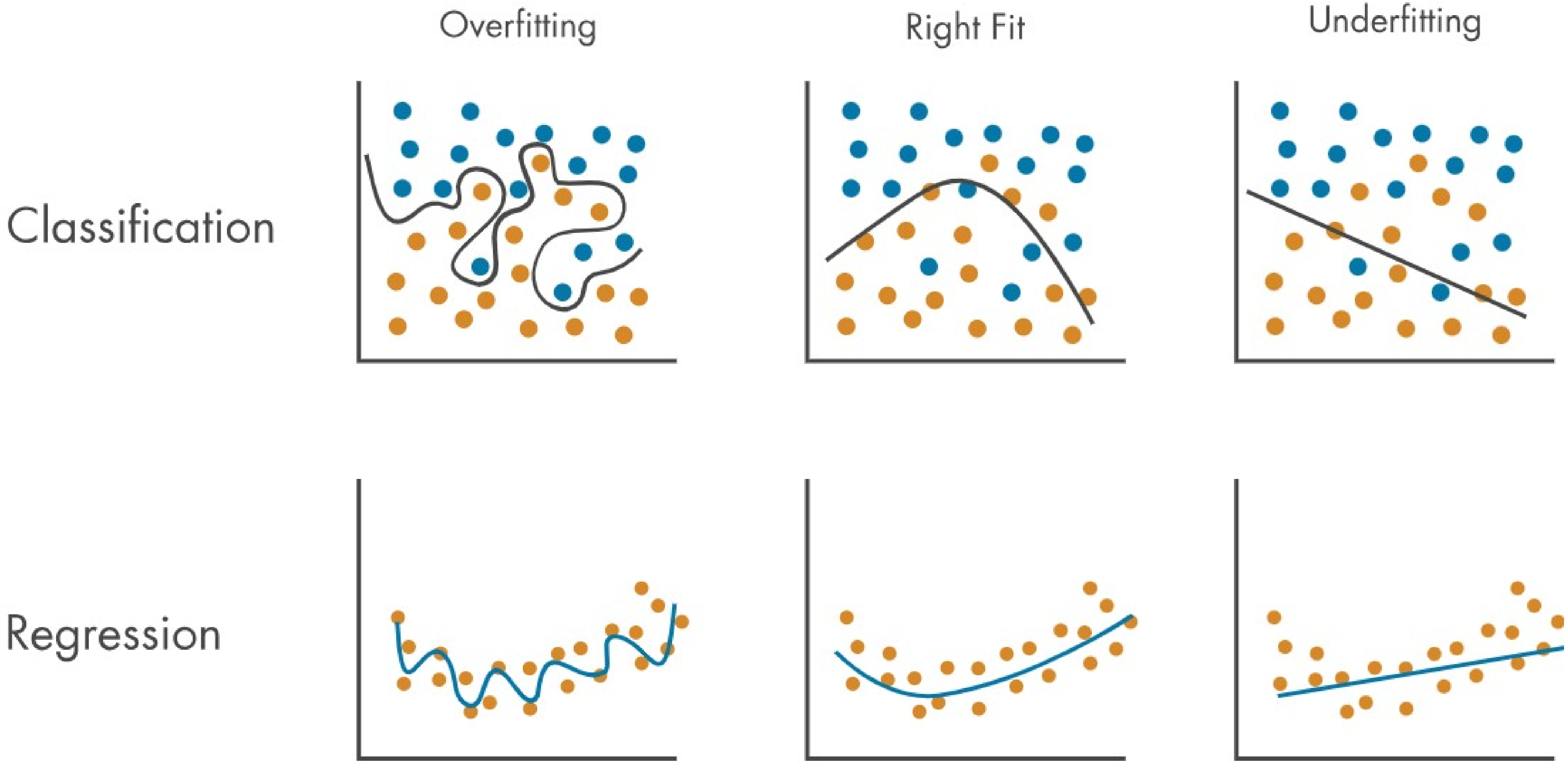
Classification



Fitting Data

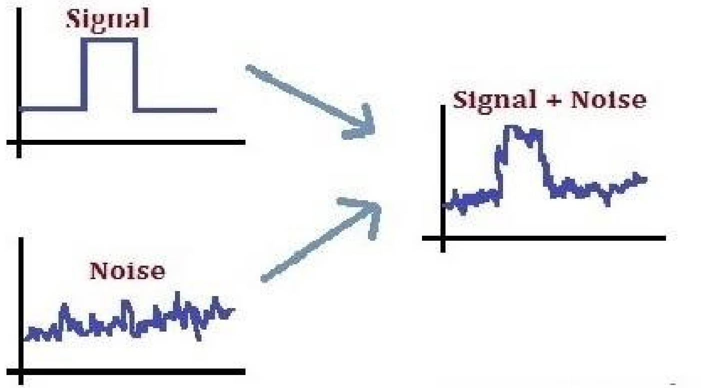


Fitting Data



Overfitting

- Overfitting: model is so closely aligned to the training data that it does not know how to respond to new data.
- Overfitting can happen because:
  - model is too complex; it memorizes very subtle patterns in the training data that don't generalize well.
  - The training data size is too small for the model complexity and/or contains large amounts of irrelevant information.



Underfitting

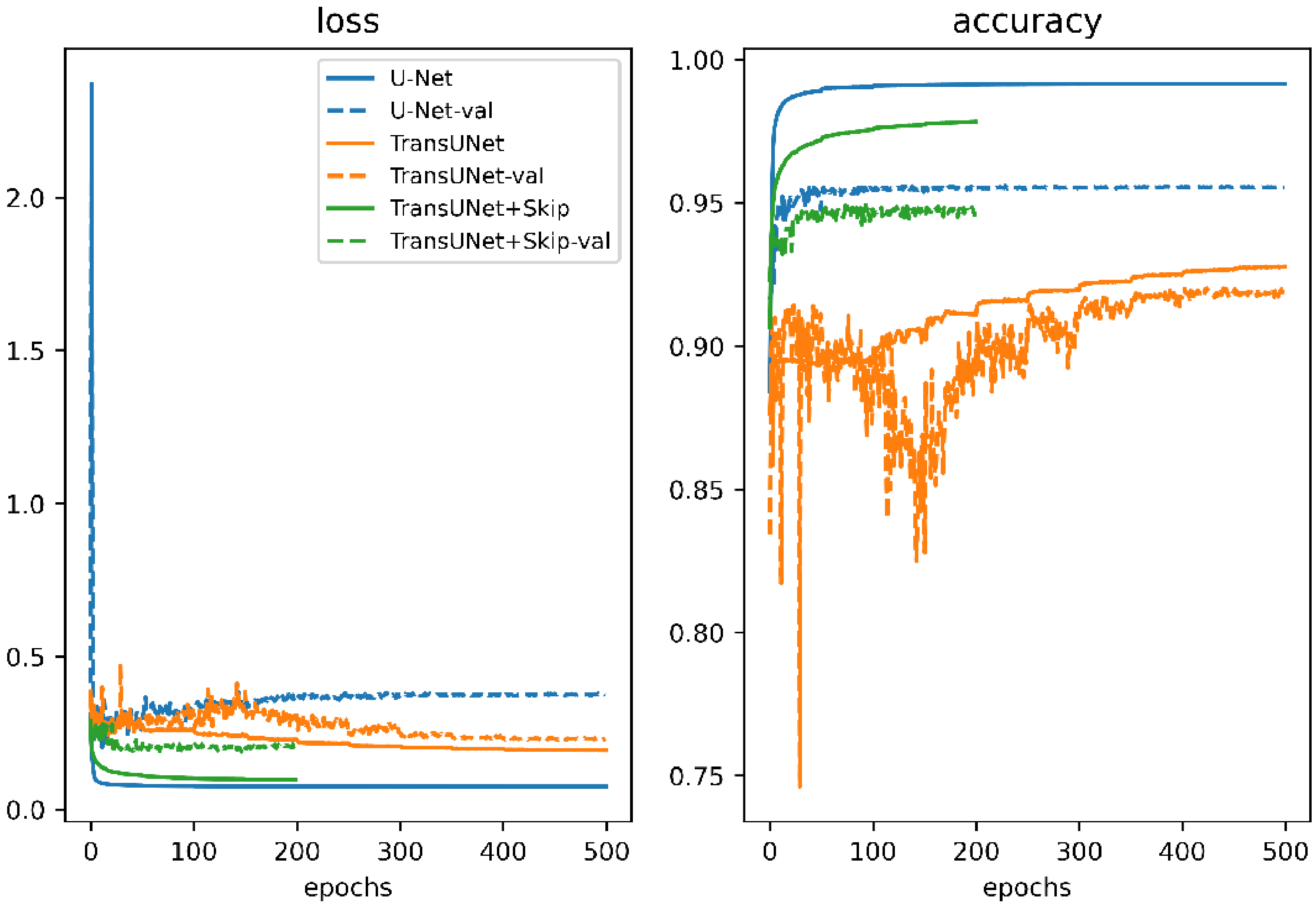
- The model couldn't see the trend
- Model is too simple to capture the underlying patterns and complexities of the data

Did We Overfit?

(loss)

Error	Overfitting	Right Fit	Underfitting
Training	[redacted]	Low	[redacted]
Test	[redacted]	Low	[redacted]

Training vs Test Loss and Accuracy



1 Epoch:
1 pass over all
training samples

Training in Real Time

```
Epoch 06 | Train loss 0.6990, Train acc 0.333, Val acc 0.800, Val AUC 0.250, F1 0.000, Precision 0.000, Recall 0.000

Train:   0%|          | 0/9 [00:00<?, ?it/s]
Train:  11%|          | 1/9 [00:00<00:06,  1.15it/s]
Train:  22%|          | 2/9 [00:01<00:05,  1.32it/s]
Train:  33%|          | 3/9 [00:02<00:04,  1.39it/s]
Train:  44%|          | 4/9 [00:02<00:03,  1.42it/s]
Train:  56%|          | 5/9 [00:03<00:02,  1.44it/s]
Train:  67%|          | 6/9 [00:04<00:02,  1.45it/s]
Train:  78%|          | 7/9 [00:04<00:01,  1.46it/s]
Train:  89%|          | 8/9 [00:05<00:00,  1.46it/s]
Train: 100%|          | 9/9 [00:06<00:00,  1.47it/s]


Val:   0%|          | 0/5 [00:00<?, ?it/s]
Val:  20%|          | 1/5 [00:00<00:00,  6.77it/s]
Val:  40%|          | 2/5 [00:00<00:00,  6.97it/s]
Val:  60%|          | 3/5 [00:00<00:00,  7.08it/s]
Val:  80%|          | 4/5 [00:00<00:00,  7.09it/s]
Val: 100%|          | 5/5 [00:00<00:00,  7.05it/s]

Epoch 07 | Train loss 0.6976, Train acc 0.333, Val acc 0.800, Val AUC 0.250, F1 0.000, Precision 0.000, Recall 0.000

Train:   0%|          | 0/9 [00:00<?, ?it/s]
Train:  11%|          | 1/9 [00:00<00:06,  1.16it/s]
Train:  22%|          | 2/9 [00:01<00:05,  1.33it/s]
Train:  33%|          | 3/9 [00:02<00:04,  1.39it/s]
Train:  44%|          | 4/9 [00:02<00:03,  1.42it/s]
Train:  56%|          | 5/9 [00:03<00:02,  1.44it/s]
Train:  67%|          | 6/9 [00:04<00:02,  1.45it/s]
Train:  78%|          | 7/9 [00:04<00:01,  1.46it/s]
Train:  89%|          | 8/9 [00:05<00:00,  1.46it/s]
Train: 100%|          | 9/9 [00:06<00:00,  1.47it/s]
```

Intuition For Fit

- Small models vs large models -> which would be more likely to overfit or underfit?

Checking External Validity

- Divide data into parts
- Train data: Data we actually use to check loss and update weights
- Test data: Data we then use to compute accuracy to see if our model works on new data

Internal v External Validity

- Internal validity: Is the model a good reflection of the data?
- External validity: Can the model be used on other data?

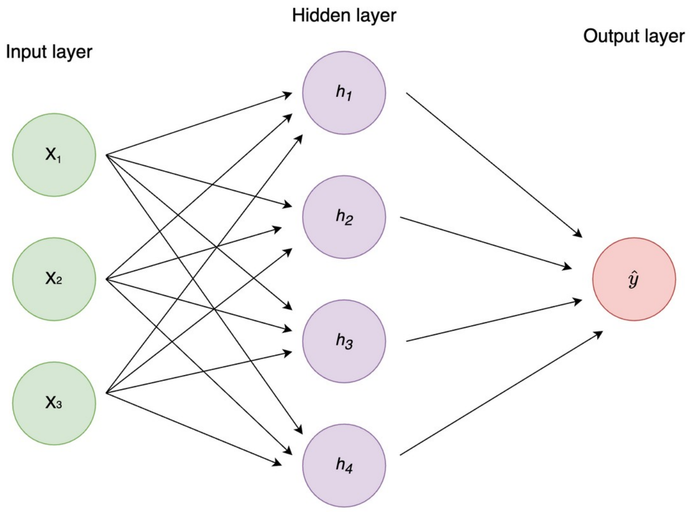
Train/Test Splitting

- What if we get a non-representative train/test split?
- Cross validation
  - Divide data into a certain number of groups
    - K of them
    - Each group is called a “fold”
  - Take turns on which group is test data, the rest are training data
  - Average accuracy across runs

Hyperparameters

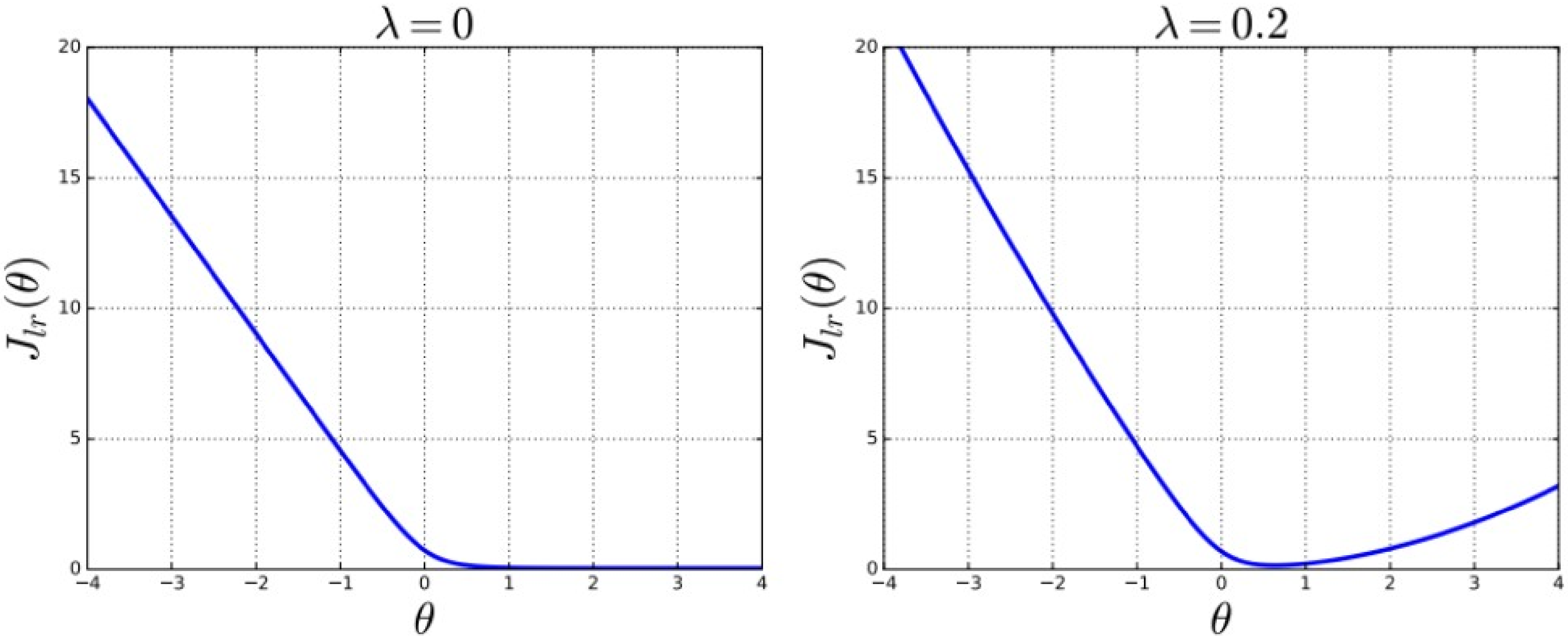
- Values that are **chosen**, not learned
- Contrast with parameters, which are the weights improved via gradient descent

Hyperparameters - Examples



- Mandatory
  - Initial weights
  - Learning rate
  - Stopping condition
- Network architecture (covered in 2 lectures)
  - Depth of the network (how many layers)
  - Thickness of the network (how many neurons in parallel per layer)
- Data related (next lecture)
  - Augmentations
- Loss function bias
  - Batch norming
  - Dropout
  - Regularization
  - Many, many more

Regularization



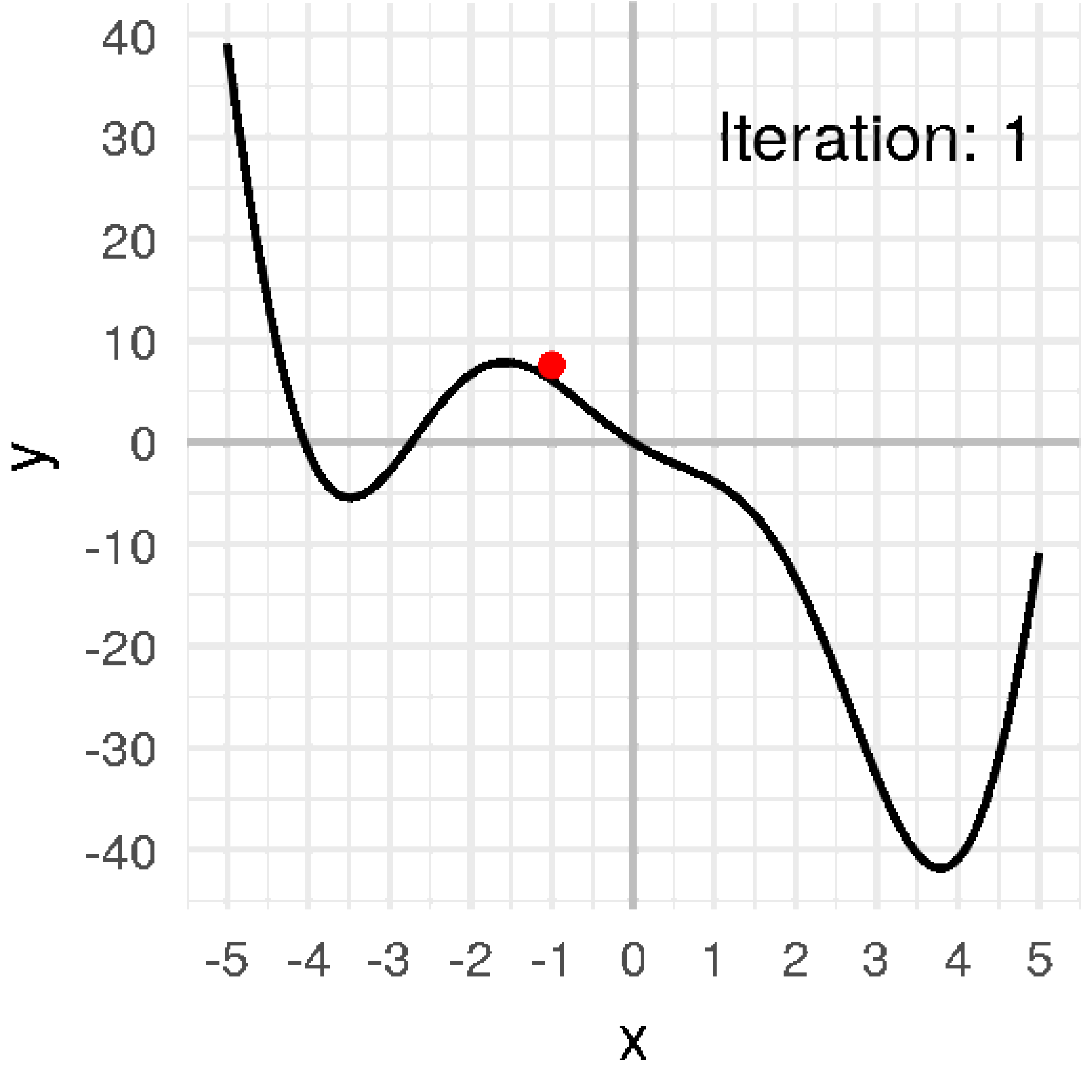
$$J_{\mathrm{lr}}(\theta, \theta_0; \mathcal{D}) = \left(\frac{1}{n} \sum_{i=1}^{n} \mathcal{L}_{\mathrm{nll}}(\sigma(\theta^T x^{(i)} + \theta_0), y^{(i)}) \right) + \lambda \left\|\theta\right\|^2$$

Ridge Regression

- Regression with a regularization term
- Lasso the same but without the square (absolute)
  - Tends to remove many parameters from the model

$$J_{\text{ridge}}(\theta, \theta_0) = \frac{1}{n} \sum_{i=1}^{n} \left(\theta^T x^{(i)} + \theta_0 - y^{(i)} \right)^2 + \lambda \|\theta\|^2$$

Gradient Descent - Revisit

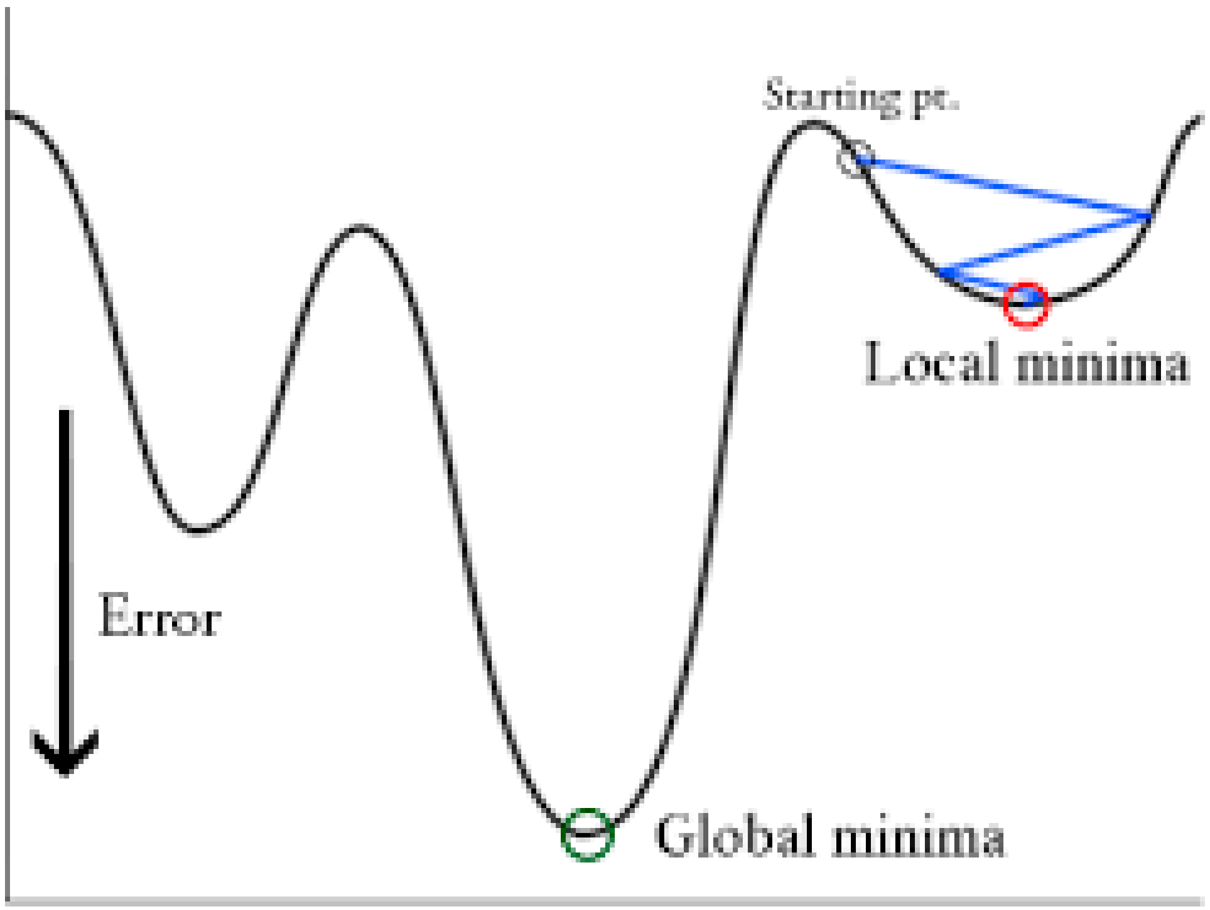


Algorithm:

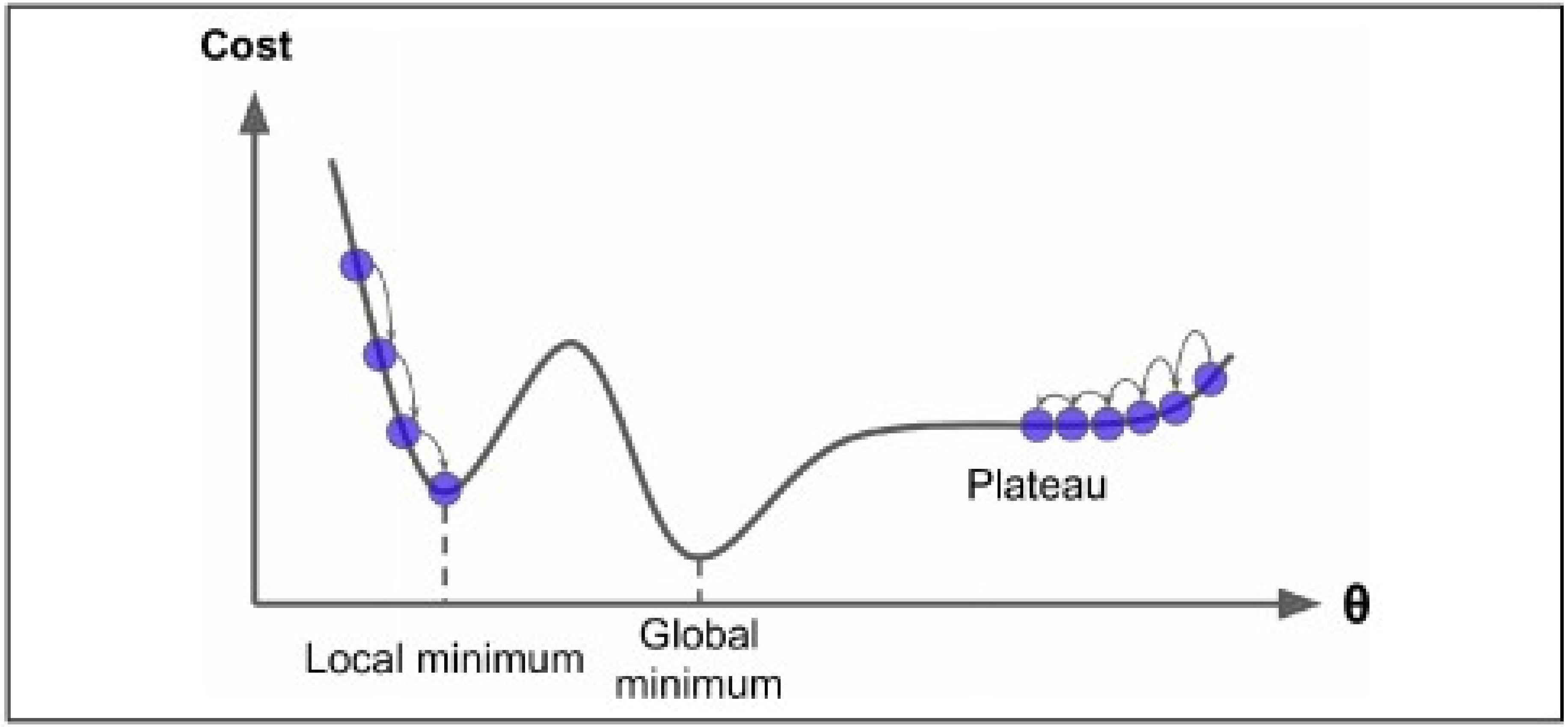
1. Find gradient at start point
2. Move opposite that direction by an amount = gradient magnitude times a constant (learning rate)
3. Repeat until the value between steps changes less than a set amount (or a fixed number of times)

$$\mathbf{a}_{n+1} = \mathbf{a}_n - \eta \nabla f(\mathbf{a}_n)$$

Gradient Descent - Revisit



Gradient Descent - Revisit



Gradient Descent - Revisit

- Takeaway: All AI algorithms are, by definition, **approximations**
- Lots of work is happening under the hood to help us find the global min/max
- Often worth retraining networks with different parameters if result isn’t good
- Choice of hyperparameters can greatly affect your loss

Stochastic GD

- Avoiding pitfalls in GD via introducing random element
- Instead of descending down gradient of all points together, choose 1 point and make weights better for just that point
- On average, processes like the normal way would

Vanishing Gradient Problem

- Over many layers, gradient can either blow up (if values large) or vanish (if values close to 0)
- Many things under the hood happening to prevent this
  - Batch norm
  - Choice of activation (ReLU safer than Sigmoid, Tanh)

Ensembling

- Methods for training multiple models on same data, then averaging or combining in smart way to get better performance
- Bootstrapping: Training many models and averaging their results
  - Build datasets by sampling w replacement, then train
  - Often used in cases with little data
- Boosting:
  - Train model, then train new model whose goal is to take output of first one and reduce its loss when applied to data
- Random Forest
  - To be discussed later

Case Study

nature > nature medicine > articles > article

Article | Open access | Published: 14 February 2025

Artificial intelligence for individualized treatment of persistent atrial fibrillation: a randomized controlled trial

Isabel Deisenhofer, Jean-Paul Albenque, Sonia Busch, Edouard Gitenay, Stavros E. Mountantonakis, Antoine Roux, Jerome Horvilleur, Babe Bakouboula, Saumil Oza, Selim Abbey, Guillaume Theodore, Antoine Lepillier, Yves Guyomar, Francis Bessiere, Jaap Jan Smit, Theophile Mohr Durdez, Paola Milpied, Anthony Appetiti, Daniel Guerrero, Tom De Potter, Christian De Chillou, Seth Goldbarg, Atul Verma, John D. Hummel & TAILORED-AF Investigators

Nature Medicine **31**, 1286–1293 (2025) | Cite this article

General Problem Statement

Measure afib recurrence in 2 groups, one informed by AI, one not

Technical Problem Statement

RCT comparing humans to classification model, which predicts regions which should be ablated

Data

Proprietary dataset of 275k (train) and 113k (test) mappings

Annotations by EP doctors in France/USA

Loss

Not listed – assume creative combo of cross entropy with DICE or other 3d loss d/t problem construction (classification+CNN)

Study aim: Compare inter observer variability to network/observer

Network

Multimodal network integrating

1. Classifier
2. Convolutional network

Training

Single train/test split

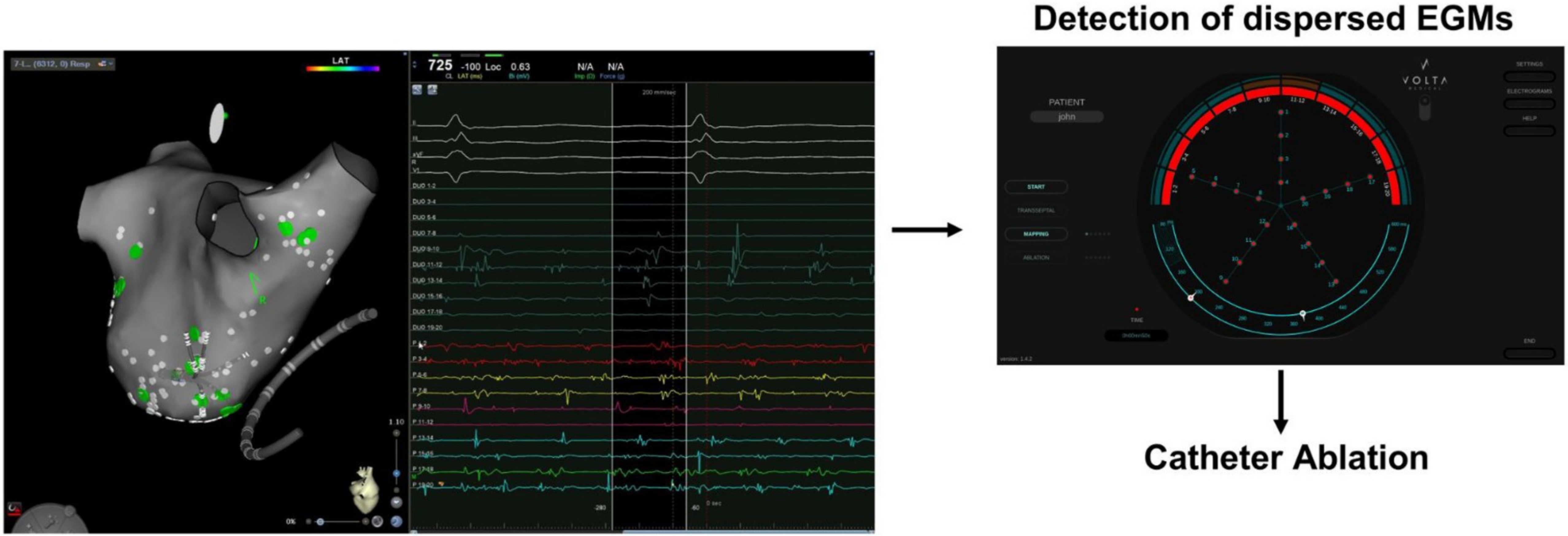
Extreme gradient boosting (ensembling)

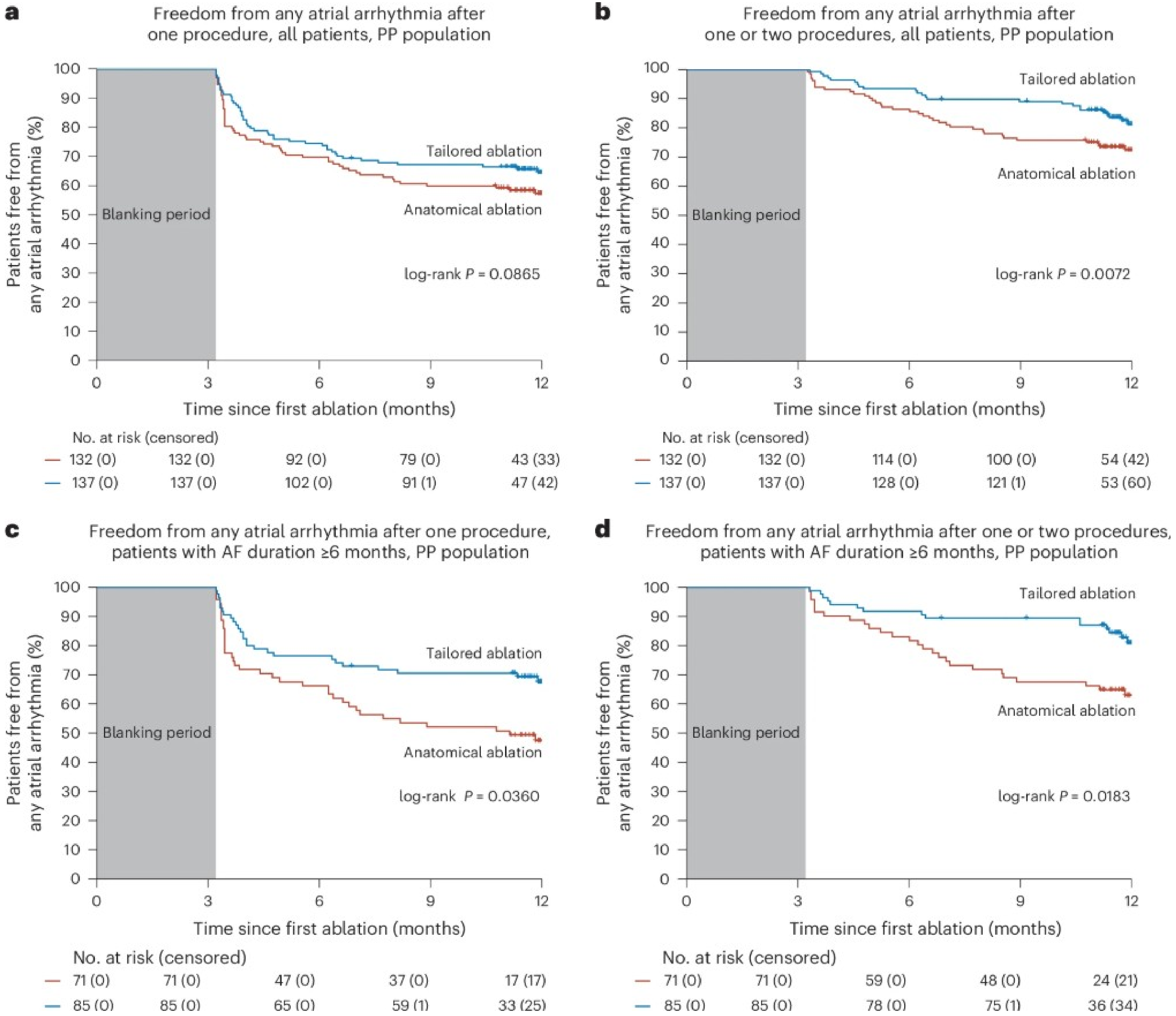
RCT Design

- 370 patients underwent catheter ablation
- Divided randomly into 2 groups: 1 with AI assistance during procedure, 1 without
  - Assistance is highlighted locations which might represent “dispersion” or sites from which electrical waves originate
  - Essentially sites of dispersion become ablation targets

Algorithm Results

- AUROC = .93 -> better when experts all agreed on location of dispersion or lack thereof




a
Freedom from any atrial arrhythmia after one procedure, all patients, PP population
Patients free from any atrial arrhythmia (%)
Blanking period
Tailored ablation
Anatomical ablation
log-rank P = 0.0865
Time since first ablation (months)
No. at risk (censored)
132 (0) 132 (0) 92 (0) 79 (0) 43 (33)
137 (0) 137 (0) 102 (0) 91 (1) 47 (42)
b
Freedom from any atrial arrhythmia after one or two procedures, all patients, PP population
Patients free from any atrial arrhythmia (%)
Blanking period
Tailored ablation
Anatomical ablation
log-rank P = 0.0072
Time since first ablation (months)
No. at risk (censored)
132 (0) 132 (0) 114 (0) 100 (0) 54 (42)
137 (0) 137 (0) 128 (0) 121 (1) 53 (60)
c
Freedom from any atrial arrhythmia after one procedure, patients with AF duration ≥6 months, PP population
Patients free from any atrial arrhythmia (%)
Blanking period
Tailored ablation
Anatomical ablation
log-rank P = 0.0360
Time since first ablation (months)
No. at risk (censored)
71 (0) 71 (0) 47 (0) 37 (0) 17 (17)
85 (0) 85 (0) 65 (0) 59 (1) 33 (25)
d
Freedom from any atrial arrhythmia after one or two procedures, patients with AF duration ≥6 months, PP population
Patients free from any atrial arrhythmia (%)
Blanking period
Tailored ablation
Anatomical ablation
log-rank P = 0.0183
Time since first ablation (months)
No. at risk (censored)
71 (0) 71 (0) 59 (0) 48 (0) 24 (21)
85 (0) 85 (0) 78 (0) 75 (1) 36 (34)

Takeaways

- Authors have clear conflict of interest
- With that said, results were significant in 2 ablation, and sustained Afib groups
  - Of course, be skeptical of subgrouping
- Noted qualitative increase in complication rate in AI assisted group
- As algorithms improve could be a useful tool

Recipe for Training

1. Problem formulation
2. Mapping between inputs and outputs
3. Choose loss function (often informed by problem formulation)
4. Train/test split
5. Train according to loss
6. Repeat training and average across many runs
7. Adjust hyperparameters as needed and repeat